

PRS Command Center — YouTube

Analytics auto-pull setup

This is the one-time setup that turns on automatic metrics pulling for the PRS Command Center. Once it's done, the per-episode and home-page metrics tiles update themselves daily from YouTube — no more copy-paste from YouTube Studio.

Roughly 30 minutes end-to-end. You'll be working in: 1. Google Cloud Console (browser) 2. Your terminal (one curl command) 3. Supabase Dashboard (SQL editor + Edge Function secrets)

The PRS YouTube channel owner needs to be involved at **Step 6** because the Analytics API is owner-scoped. They sign in once and the refresh token lives in the database from then on.

Prerequisites

The static page assets are already deployed. Before starting:

- [] Run `prs/sql/migrate_2026_05_26_video.sql` in Supabase SQL editor (adds `video_url` + `video_id` to `prs_episodes`).
- [] Run `prs/sql/setup_oauth_tokens.sql` in Supabase SQL editor (creates the `prs_oauth_tokens` table; RLS locked down to `service_role` only).

Step 1 — Google Cloud project

1. Go to <https://console.cloud.google.com>.
2. Top bar → project selector → **New Project**. Name it `prs-command-center` (or reuse an existing project).
3. Make sure billing is **not** required — both the APIs we use have free quotas that cover us comfortably.

Step 2 — Enable the two APIs

In your new project:

1. **APIs & Services → Library**.
2. Search for and enable **YouTube Data API v3**.
3. Search for and enable **YouTube Analytics API**.

Both are needed: the Data API is what resolves channel/video info; the Analytics API is what serves the metrics.

Step 3 — Create the OAuth 2.0 client

1. **APIs & Services** → **OAuth consent screen**.
2. User type: **External**. Click *Create*.
3. App name: `PRS Command Center`. User support email: yours. Developer contact: yours. Save and continue.
4. Scopes: skip (we'll request scopes at auth time).
5. Test users: add the **PRS YouTube channel owner's Google email**. This is the only account that needs to authorize. Add yours too if you're testing from a different account.
6. Save → back to dashboard.

Then:

1. **APIs & Services** → **Credentials** → + **Create credentials** → **OAuth client ID**.
2. Application type: **Web application**.
3. Name: `PRS auto-sync`.
4. **Authorized redirect URI**: `https://giddyuptiger.github.io/prs/oauth-callback.html`
5. *Create*. Google shows you a **Client ID** and **Client secret** — copy both. You'll need them in Steps 5 and 6.

Step 4 — Find the PRS YouTube channel ID (*optional but recommended*)

If the channel owner's Google account only manages one YouTube channel, you can skip this — the function defaults to `MINE` which means "whichever channel the token owner manages."

If they manage multiple, find the specific channel ID:

1. Go to `https://www.youtube.com` → click the profile picture → **Your channel** → URL shows `youtube.com/channel/UCxxxxxxxxxxxxxx`.
2. That `UCxxxx...` string is the channel ID.

Step 5 — Set the Edge Function secrets

In Supabase Dashboard → your project → **Edge Functions** → (if the function isn't deployed yet, see `prs/functions/youtube-analytics-sync/README.md`) → **Manage secrets**, set:

Name	Value
YT_CLIENT_ID	Client ID from Step 3.
YT_CLIENT_SECRET	Client secret from Step 3.
YT_CHANNEL_ID	The UCxxxx... from Step 4 — or omit / set to MINE .

SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY are auto-injected by Supabase; you don't set them.

Step 6 — One-time OAuth authorization (*channel owner only*)

The PRS YouTube channel owner does this part. Send them this URL — fill in YT_CLIENT_ID_HERE with the Client ID from Step 3 first:

```
https://accounts.google.com/o/oauth2/v2/auth?response_type=code&access_type=offline&prompt=consent
```

When they:

1. Visit the URL, they'll be prompted to sign in (must be the channel owner account).
2. They'll see a Google consent screen asking for **YouTube Analytics (read-only)** and **YouTube (read-only)** access. They click *Allow*.
3. Google redirects them to `https://giddyuptiger.github.io/prs/oauth-callback.html?code=...`.
4. That page shows the authorization code and the two follow-up commands. The owner copies the code (or hands the whole URL back to you — the code is in the `?code=` query param).

Step 7 — Exchange the code for a refresh token

In your terminal:

```
export YT_CLIENT_ID="...from Step 3..."
export YT_CLIENT_SECRET="...from Step 3..."
export AUTH_CODE="...from Step 6..."

curl -X POST "https://oauth2.googleapis.com/token" \
  -d "code=$AUTH_CODE" \
  -d "client_id=$YT_CLIENT_ID" \
  -d "client_secret=$YT_CLIENT_SECRET" \
  -d "redirect_uri=https://giddyuptiger.github.io/prs/oauth-callback.html" \
  -d "grant_type=authorization_code"
```

The response is JSON like:

```
{
  "access_token": "ya29....",
  "expires_in": 3599,
  "refresh_token": "1//0g...",
  "scope": "https://www.googleapis.com/auth/yt-analytics.readonly https://www.googleapis.com/auth/youtube",
  "token_type": "Bearer"
}
```

Copy the `refresh_token` string. That's the one piece of long-lived credential. The `access_token` is short-lived (1 hour); the function refreshes it on every run.

⚠ If you see `"error": "invalid_grant"` — the auth code is single-use and expires in ~10 minutes. Re-do Step 6 to get a fresh code.

Step 8 — Store the refresh token in Supabase

Open the Supabase SQL editor and run:

```
insert into prs_oauth_tokens (provider, refresh_token, scope)
values (
  'google',
  'PASTE_REFRESH_TOKEN_HERE',
  'https://www.googleapis.com/auth/yt-analytics.readonly https://www.googleapis.com/auth/youtube'
);
```

You can re-run this if the token ever needs to be replaced — the edge function reads the most recent row (`order by updated_at desc limit 1`), so just `insert` a new one, no need to delete the old.

Step 9 — Test the function

```
curl -X POST "https://umtckshrjeyueyzgkuwm.supabase.co/functions/v1/youtube-analytics-sync" \
-H "Authorization: Bearer $SUPABASE_ANON_KEY"
```

You should see something like:

```
{
  "ok": true,
  "ran_at": "2026-05-26T20:00:00.000Z",
  "synced": 9,
  "total": 9,
  "results": [
    { "slug": "rob-morrow", "ok": true, "views": 12345 },
    ...
  ]
}
```

```
]
}
```

Open the PRS Command Center home page — the metrics tiles on each card should now reflect YouTube's numbers, and `metrics_updated_at` is the run timestamp.

If `total: 0`, no episodes have a `video_id` set yet. Open any episode in the dashboard, click **Video** in the metrics card header, paste the YouTube URL, and re-run. The auto-sync uses `video_id`, which the page derives from `video_url` on save.

Step 10 — Schedule the daily cron

Two options:

Option A — Supabase Dashboard cron (*recommended*)

1. Supabase Dashboard → **Edge Functions** → **Cron Jobs** → *New job*.
2. Function: `youtube-analytics-sync`.
3. Schedule: `0 11 * * *` (11:00 UTC = 04:00 PT, after the YouTube Analytics daily rollover).
4. Save.

Option B — `pg_cron` from Postgres

```
-- One-time setup of pg_cron + http extension if not already enabled:
create extension if not exists pg_cron;
create extension if not exists pg_net; -- exposes net.http_post()

-- Schedule (UTC). Replace ${SUPABASE_ANON_KEY} inline.
select cron.schedule(
  'prs-yt-sync-daily',
  '0 11 * * *',
  $$
  select net.http_post(
    url := 'https://umtckshrjeyueyzgkuwm.supabase.co/functions/v1/youtube-analytics-sync',
    headers := jsonb_build_object(
      'Authorization', 'Bearer ${SUPABASE_ANON_KEY}',
      'Content-Type', 'application/json'
    ),
    body := '{}'::jsonb
  );
  $$
);
```

To remove later: `select cron.unschedule('prs-yt-sync-daily');`

What happens next

- Every day at 04:00 PT the function runs, pulls metrics for every episode with a `video_id`, and writes them into `prs_episodes.metrics`.
- The PRS Command Center reads from Supabase as before — no code change needed. Metrics tiles auto-populate.
- Manual edits via the in-app **Edit** button still work. Three fields always stay manual because they live in Google Ads, not YouTube: `ad_spend`, `ad_ctr_pct`, `ad_cpv`.
- If a video is deleted or made private, the sync result for that row will be `{ ok: false }` with an error message — the rest of the episodes still sync.

What to do if it stops working

The most common failure is the refresh token getting revoked (e.g., the channel owner changed their Google password, or the OAuth consent screen is still in test mode and the test-user grant expired after 7 days).

Fix: 1. Re-run Steps 6 → 8 with the same Client ID / Secret to get a new refresh token. 2. `insert into prs_oauth_tokens ...` again — the function picks up the newest row.

To stop the test-user 7-day expiry permanently, publish the OAuth consent screen (APIs & Services → OAuth consent screen → *Publish app*). Google will email you about it; for an internal/self-use app with one authorized account, the review is trivial.